# Velora Cinema

**Name : Aarika**
**ID:2024A7PS0023H**
**Name: Chaitrali Shrikant Dharmadhikari**
**ID: 2024A7PS0133H**
**Name: Nivedita Mahesh**
**ID: 2024A7IS2425H**
**Name: Ananya Joshi**
**ID: 2024A7IS2408H**
**Name: Sejal Mittal**
**ID: 2024A7PS0161H**
**Name: Riya Anilkumar Vaghani**
**ID: 2024A7IS0109H**

## 1. Problem Description and Assumptions

The Online Movie Booking System is designed to manage and automate the complete workflow of movie ticket booking across multiple cities and theatres.

The system handles:

- City-wise theatre management
- Multiple screens per theatre
- Movie scheduling and show timings
- Seat categorization and dynamic pricing
- User bookings and seat locking
- Online payment processing
- Movie reviews and ratings
- Genre management

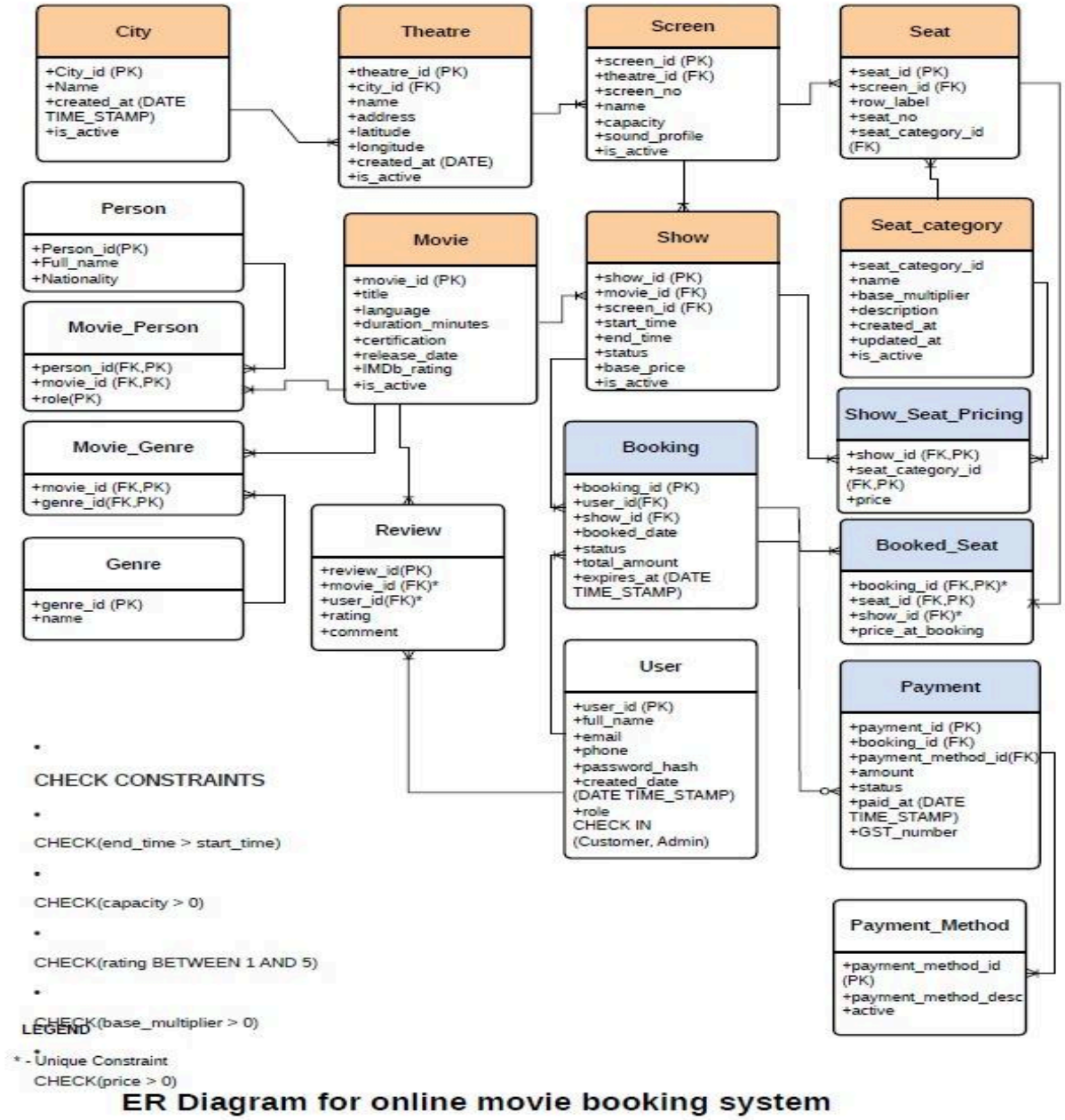
Unlike a simple booking system, this model supports:

- Dynamic seat pricing per show
- Review and rating system
- Role-based users (Customer / Admin)

## Assumptions

1. A City can have multiple theatres.
2. A **Theatre** belongs to exactly one city.
3. A **Theatre** can have multiple screens.
4. A **Screen** belongs to exactly one theatre.
5. A **Screen** has multiple seats.
6. Each seat belongs to one seat category .
7. A **Show** is scheduled for one movie on one screen at one particular time i.e.
8. A show has a base price and seat-category-wise multipliers.
9. A **User** can make multiple bookings.
10. Each User can review a movie only once.
11. A booking belongs to exactly one show.
12. A booking can include multiple seats.
13. A seat can be booked only once per show.
14. A booking must be paid before confirmation otherwise it expires in ten minutes automatically.
15. Only Admin users can manage movies, theatres, screens, and shows.
16. Soft delete is implemented using the attribute is_active.

## 2. ER Diagram




ER Diagram for online movie booking system

## 3. Constraint Specification

### Entity Integrity Constraints (Primary Keys)

Every entity must have a **unique and NOT NULL primary key**.

## Primary Keys from Your ER:

- `City(city_id)`
- `Theatre(theatre_id)`
- `Screen(screen_id)`
- `Seat(seat_id)`
- `Seat_Category(seat_category_id)`
- `Movie(movie_id)`
- `Genre(genre_id)`
- `Person(person_id)`
- `Show(show_id)`
- `User(user_id)`
- `Booking(booking_id)`
- `Payment(payment_id)`
- `Payment_Method(payment_method_id)`
- `Review(review_id)`

## Composite Primary Keys:

- `Movie_Genre(movie_id, genre_id)`
- `Movie_Person(person_id, movie_id)`
- `Show_Seat_Pricing(show_id, seat_category_id)`
- `Booked_Seat(booking_id, seat_id)`

## Referential Integrity Constraints (Foreign Keys)

### City – Theatre

- `Theatre.city_id → City.city_id`

### Theatre – Screen

- `Screen.theatre_id → Theatre.theatre_id`

### Screen – Seat

- `Seat.screen_id → Screen.screen_id`

### Seat – Seat_Category

- `Seat.seat_category_id → Seat_Category.seat_category_id`

### Movie Relationships

- `Movie_Genre.movie_id → Movie.movie_id`
- `Movie_Genre.genre_id → Genre.genre_id`
- `Movie_Person.movie_id → Movie.movie_id`
- `Movie_Person.person_id → Person.person_id`

### Show Relationships

- `Show.movie_id → Movie.movie_id`
- `Show.screen_id → Screen.screen_id`

### Booking Relationships

- `Booking.user_id → User.user_id`
- `Booking.show_id → Show.show_id`

### Booked Seat

- `Booked_Seat.booking_id → Booking.booking_id`
- `Booked_Seat.seat_id → Seat.seat_id`
- `Booked_Seat.show_id → Show.show_id`

### Payment

- `Payment.booking_id → Booking.booking_id`
- `Payment.payment_method_id → Payment_Method.payment_method_id`

### Review

- `Review.movie_id → Movie.movie_id`
- `Review.user_id → User.user_id`

## <u>Domain Constraints (CHECK Constraints)</u>

These restrict valid attribute values.

- **Show Time Constraint**

CHECK(end_time > start_time)

- **Screen Capacity Constraint**

  CHECK(capacity > 0)

- **Review Rating Constraint**

  CHECK(rating BETWEEN 1 AND 5)

- **Seat Pricing Constraint**

  CHECK(price > 0)

- **Base Multiplier Constraint (Seat Category Pricing)**

  CHECK(base_multiplier > 0)

## Key Constraints (Uniqueness Constraints)

- **Unique Review per User per Movie**

  One user can review a movie only once:

  UNIQUE(user_id, movie_id)

- **Seat cannot be booked twice for same show**

  UNIQUE(show_id, seat_id)

- **Unique Screen Number per Theatre**

  UNIQUE(theatre_id, screen_no)

- **Unique Seat Number per Screen**

  UNIQUE(screen_id, seat_no)

## Business Rule Constraints (Application-Level Constraints)

These are logical constraints derived from system behavior:

- A booking cannot be confirmed unless payment status = 'Successful'.

- Seats must belong to the same screen as the show being booked.
- A show must not overlap with another show on the same screen at the same time.
- A show can only be created for active movies and active screens.
- Users must have role either:
    - 'Customer'
    - 'Admin'
- Expired bookings (expires_at) automatically release seats.
- Seat pricing for a show must exist before booking is allowed.

### Participation Constraints (From ER Relationships)

### Total Participation:

- A `Screen` must belong to a `Theatre`.
- A `Show` must belong to a `Movie` and `Screen`.
- A `Booking` must belong to a `User` and a `Show`.

### Partial Participation:

- A Movie may or may not have reviews.
- A User may or may not make bookings.

# 4. Frontend

The frontend is logically organized into modules that directly correspond to database-level CRUD operations.

### Registration (INSERT Operation)

A structured form captures:

- Full name
- Email
- Phone number
- Password

On submission:

- A new user record is created in the `users` table.
- Form validation ensures correct input formats before API submission.

### Login (Credential Exchange)

- Accepts user credentials via secure form submission.
- Exchanges credentials for a JWT Access Token.
- Enables authenticated operations across the application.

## Movie Retrieval (SELECT Operation)

- Fetches and displays all available movies.
- Displays essential identifiers to assist in referencing during booking or scheduling.
- Acts as a bridge between user browsing and database records.

## Booking Engine (INSERT & UPDATE Operations)

### Create Booking (INSERT)

- Links a user to a selected show.
- Associates specific seat IDs with the booking.
- Writes a new booking record into the database.

### Cancel Booking (UPDATE)

- Modifies the `status` field of an existing booking (e.g., sets to *Cancelled*).
- Demonstrates controlled state transitions rather than hard deletion.
- Maintains auditability and data consistency.

## Admin Command Center

The Admin module serves as a controlled interface for high-level database operations.

### Movie Onboarding

A comprehensive form for inserting detailed cinematic metadata including:

- IMDb ratings
- ISO-formatted release dates
- Certification details
- Language and runtime

This ensures structured and consistent data entry into the system.

### Show Scheduling

- Maps movies to specific screens
- Assigns show timings
- Connects relational entities (movie → screen → time slot)

This interface demonstrates relational database integration through practical UI interactions.

### Real-Time Debug Console (Advanced Feature)

One of the standout features of CineBook is the integrated **Debug Log Console**.

Purpose:

- Captures every API request and response
- Displays raw JSON payloads
- Shows HTTP status codes and server responses

### Demonstration Workflow (Live Presentation Flow)

The frontend is optimized for a structured 7-step demo sequence:

- **Verify Backend Connectivity**
  Confirm API server is running.
- **Register a New User**
  Demonstrate INSERT into the users table.
- **Login to Authenticate**
  Retrieve and store JWT token.
- **Fetch Movie Records**
  Demonstrate SELECT operation.
- **Create a Booking**
  Insert booking and booked_seat records.
- **Cancel a Booking**
  Update booking status (state transition).
- **Inspect Debug Console**
  Show raw JSON requests/responses to validate system correctness.

# 5. Steps of Creating the System

The development of the system followed a structured approach.

1. First, the problem requirements were analyzed to identify the main operations such as browsing movies, selecting shows, booking seats, making payments, and managing administrative tasks.
2. Based on these requirements, the key entities of the system such as City, Theatre, Screen, Seat, Movie, Show, Booking, Payment, and Review were identified.
3. An Entity-Relationship (ER) diagram was then designed to represent the relationships between these entities. The ER diagram was converted into a relational schema by defining tables, primary keys, foreign keys, and necessary constraints.
4. The database was implemented in PostgreSQL using SQL scripts.

5. After the database design, the backend was developed using FastAPI to create REST API endpoints for operations such as retrieving movies, viewing shows, booking seats, processing payments, and managing reviews. Additional constraints such as preventing overlapping shows and double seat bookings were implemented using database triggers and backend validation.
6. Finally, a frontend interface was developed to allow users to interact with the system, and the entire system was tested to ensure correct functionality and integration between the database, backend, and frontend components.